Niveau Master M2 DEML

PROJET FINAL DU MODULE :

MLOps II et automatisation de processus de déploiement de modèles ML sur le cloud

Pipeline MLOps Complet — Application IA en Production

Établissement	Institut Supérieur du Numérique (SupNum)
Filière	Master DEML M2 – Data Engineering & Machine Learning
Cours	MLOps II – Automatisation du déploiement de modèles ML
Encadrant	Professeur Yehdhih ANNA
Groupes	4 étudiants par groupe — liste à soumettre avant ce samedi
Date Limite	25 Avril 2025 (Date des présentations).
Notation	**Note sur 20 points (grille stricte)**

⚠️ **À faire avant ce samedi**

Une liste de groupe consolidée (4 personnes par groupe) avec les matricules. Doit etre soumis avant ce samedi (11 avril 2026)

Un seul repository GitHub par groupe — à créer immédiatement avec la structure de branches décrite dans ce document.

Une seule instance EC2 par groupe (t3.medium ou t2.medium) — mutualisez les ressources.

⚠️ **NB : Ceci n'est pas un projet de dernières minutes**

La charge de travail dans ce projet n'est pas à sous-estimer, prenez le relais immédiatement, pour ne pas être stresser par les petits détails du projet au dernier minutes.

1. Contexte et Objectif du Projet

Ce projet est la synthèse de tout ce que vous avez appris dans les trois laboratoires du cours MLOps II. Il vous demande de concevoir et déployer un pipeline MLOps complet et autonome autour d'un problème de machine learning de votre choix.

Contrairement aux labs qui guidaient chaque étape, ce projet vous place en situation réelle : vous choisissez le problème, les données, le modèle, et vous construisez l'intégralité de l'infrastructure. L'évaluation porte autant sur la rigueur de l'implémentation que sur la qualité du pipeline automatisé.

🎯 **Ce que ce projet simule**

Un ingénieur MLOps en entreprise doit livrer un modèle en production et s'assurer que tout changement — code ou données — déclenche automatiquement un nouveau cycle d'entraînement, d'évaluation et de déploiement.
C'est exactement ce que vous devez construire.

1.1. Liberté de choix du sujet

Vous êtes libres de choisir n'importe quel problème de machine learning supervisé ou non supervisé, tant que le pipeline MLOps complet est respecté. Quelques exemples non-exhaustifs :

🚫 **Contrainte absolue**

Il est interdit d'utiliser le dataset diabetes.csv des labs. Choisissez un dataset différent.
Le dataset doit être stocké sur S3 comme source de données primaire (simulant un ETL externe).

2. Livrables Attendus

Le projet est composé de six livrables distincts, chacun évalué indépendamment. L'absence d'un livrable entraîne automatiquement zéro sur la section correspondante.

2.1. Serveur MLflow sur EC2 avec Artefacts sur S3

Un serveur MLflow doit être déployé sur une instance EC2 et rester accessible pendant la démonstration. Tous les artefacts (modèles, métriques, paramètres) doivent être persistés sur un bucket S3.

- Instance EC2 : t3.medium ou t2.medium (Ubuntu), une seule par groupe.
- Bucket S3 : contient le cache MLflow (runs, expériences, artefacts loggués, modèles).
- Le serveur doit être accessible sur le port `5000` depuis l'extérieur.
- Le serveur doit tourner en arrière-plan (via `nohup` ou équivalent) sans nécessiter une session SSH ouverte.
- Le modèle entraîné doit être enregistré dans le MLflow Model Registry sous un nom explicite.

> 💡 **Rappel technique**
> Voir Lab 1 sections 1.1 pour la configuration complète.
> Le MLflow Registry est l'endroit où l'API de serving va chercher le 'latest' modèle au démarrage.

2.2. Pipeline ML Automatisé avec DVC

Le circuit machine learning complet doit être structuré en stages DVC, déclenchables en une seule commande. DVC garantit la reproductibilité et le versioning des données.

Le pipeline doit contenir au minimum les stages suivants, dans cet ordre :

Stage	Description attendue
preprocess	Chargement des données depuis S3, nettoyage, encodage, normalisation, split train/test. Sauvegarde de la version preprocessée.
train	Entraînement du modèle sur les données preprocessées. Logging des hyperparamètres et métriques dans MLflow.
evaluate	Évaluation sur le jeu de test. Logging des métriques finales (accuracy, F1, AUC, etc.) dans MLflow.
register	Enregistrement du modèle dans le MLflow Model Registry si les métriques dépassent un seuil défini.

- La source de données doit être déclarée avec `dvc import-url` pointant vers votre fichier sur S3.
- Le pipeline complet doit se relancer avec `dvc repro`.
- Le remote DVC doit être configuré sur S3 pour stocker le cache de versioning.

2.3. API de Prédiction et Interface Utilisateur

L'application doit exposer une API REST et une interface utilisateur permettant à un utilisateur non-technique d'interagir avec le modèle.

API (FastAPI , Flask, Djangon ou autres …)

- L'API doit exposer au minimum un endpoint `POST /predict` qui accepte les features en JSON et retourne la prédiction.
- Un endpoint `GET /` (health check) doit retourner un statut de disponibilité de l'API.
- L'API doit charger automatiquement le dernier modèle enregistré dans le MLflow Registry au démarrage.
- L'API doit être conteneurisée avec Docker et exposée sur le port 8000.

Interface Utilisateur

- Une interface web (HTML/CSS/JS, React, ou tout framework frontend) doit permettre de saisir les features du modèle et d'afficher la prédiction.

- L'interface peut être servie par l'API elle-même (fichiers statiques) ou être un frontend séparé.
- L'interface doit être fonctionnelle et accessible depuis un navigateur — pas uniquement testable via curl.

> 💡 **Conseil d'implémentation**
> Vous pouvez servir l'interface directement depuis votre backend avec StaticFiles ou en intégrant le HTML dans un endpoint GET /.

2.4. Pipeline CI/CD Automatisé

Deux workflows d'automatisation distincts doivent être configurés. GitHub Actions est recommandé, mais tout outil équivalent (GitLab CI, Jenkins, CircleCI…) est accepté, tant que les déclencheurs et comportements attendus sont respectés.

Workflow 1 — Déclencheur : Merge sur main (Production)

- Ce workflow se déclenche à chaque **merge vers la branche main**.
- Il doit : se connecter à l'EC2 via SSH, mettre à jour le code (git pull), reconstruire l'image Docker, et redémarrer le conteneur API.
- Un **health check** automatique doit vérifier que l'API répond après le déploiement.
- Les credentials AWS et la clé SSH doivent être stockés en tant que secrets CI/CD (GitHub Secrets ou équivalent) — jamais dans le code.

Workflow 2 — Déclencheur : Données (au choix du groupe)

- Ce workflow doit relancer dvc repro (ou équivalent) pour entraîner un nouveau modèle à partir des données mises à jour.
- Le déclencheur est libre : cron planifié, déclenchement manuel (workflow_dispatch), push sur une branche de données, ou détection de modification de fichier .dvc.
- Après le retraining, le nouveau modèle doit être enregistré dans MLflow et l'API doit être redémarrée avec ce nouveau modèle.

> 🔐 **Règle de sécurité obligatoire**
> Aucun secret (clé AWS, clé SSH, mot de passe) ne doit apparaître dans le code source, les fichiers de configuration, ou l'historique git.
> Tout secret trouvé dans le code entraîne une pénalité de -2 points sur la note finale.

2.5. Structure Git et Collaboration

La gestion du code source et la collaboration de l'équipe sont évaluées à travers la structure et l'historique du repository GitHub.

Structure des branches obligatoire

Branche	Rôle et règles
main	Branche de production. Protégée — ne reçoit que des merges depuis staging. Chaque merge déclenche le Workflow CI/CD 1.
staging	Branche d'intégration. Reçoit les merges des branches dev individuelles avant validation.
dev-MATRICULE	Une branche par étudiant (ex: dev-21001). Chaque membre doit avoir des commits sur sa branche dans l'historique.

- Chaque membre du groupe doit avoir au moins 3commits significatifs (pas uniquement des commits de merge) dans l'historique git.
- Les messages de commit doivent être descriptifs et en rapport avec le travail effectué.
- params.yaml et tout fichier contenant des secrets doivent être dans le .gitignore.

2.6. Infrastructure AWS et Optimisation des Ressources

- Une seule instance EC2 (t3.medium ou t2.medium) par groupe. Les deux services (MLflow :5000 et API Docker :8000) tournent sur la même machine.
- Un seul bucket S3 par groupe, organisant : les données raw, les données preprocessées, le cache MLflow, et le store DVC.
- Si vous manquez d'espace disque sur l'EC2, augmentez le volume EBS de +10 GB via la console AWS (**Actions → Modify Volume**).
- Les ports 22, 5000 et 8000 doivent être ouverts dans le Security Group.

3. Grille de Notation Détaillée

La note finale est sur 20 points : 19 points selon la grille ci-dessous, plus 1 point d'assiduité attribué individuellement. La grille est stricte — chaque sous-critère est évalué en binaire (obtenu / non obtenu) ou en demi-points.

📌 **Important**

La note du projet est une note de groupe : tous les membres reçoivent la même note de base.
Le point d'assiduité est individuel : il est attribué aux étudiants les plus assidus, et peut générer un bonus de +0.5 sur la note de groupe si la majorité du groupe est concernée.

A — Infrastructure MLflow & AWS (3 points)

Critère	Détail d'évaluation	Points
A1 — Serveur MLflow	Serveur MLflow démarré sur EC2, accessible sur port 5000, tourne en arrière-plan (nohup/screen) sans session SSH ouverte.	1 pt
A2 — Persistance S3	Artefacts MLflow (runs, modèles, expériences) bien stockés sur S3. Bucket	1 pt

	correctement configuré comme default-artifact-root.	
A3 — Model Registry	Au moins un modèle enregistré dans le MLflow Model Registry avec un nom explicite, une version et un statut (Staging/Production).	1 pt

B — Pipeline DVC (4 points)

Critère	Détail d'évaluation	Points
B1 — Source de données	Dataset déclaré avec dvc import-url depuis S3. Fichier .dvc présent et versionné dans git.	0.5 pt
B2 — Stage preprocess	Stage DVC fonctionnel : lecture depuis S3, preprocessing (nettoyage, encodage, normalisation), sauvegarde du résultat.	1 pt
B3 — Stage train	Stage DVC fonctionnel : entraînement du modèle, logging des hyperparamètres et métriques dans MLflow.	1 pt
B4 — Stage evaluate	Stage DVC fonctionnel : évaluation sur jeu de test, logging des métriques finales dans MLflow.	0.5 pt
B5 — dvc repro	La commande dvc repro exécute l'intégralité du pipeline de bout en bout sans erreur.	0.5 pt
B6 — Remote DVC S3	Remote DVC configuré sur S3, dvc push et dvc pull fonctionnels.	0.5 pt

C — API et Interface Utilisateur (4 points)

Critère	Détail d'évaluation	Points
C1 — Endpoint /predict	POST /predict accepte les features en JSON, retourne la prédiction correctement. Testé avec curl ou Postman.	1 pt
C2 — Health check	GET / retourne un message de statut et un code HTTP 200. Utilisé par le workflow CI/CD.	0.5 pt
C3 — Chargement modèle	L'API charge automatiquement le 'latest' modèle depuis MLflow Registry au démarrage du conteneur.	1 pt
C4 — Conteneurisation	Dockerfile valide, image construite sans erreur, conteneur lancé avec les variables d'environnement AWS et MLflow.	0.5 pt
C5 — Interface utilisateur	Interface web accessible depuis un navigateur, permettant la saisie des features et l'affichage de la prédiction. Fonctionnelle end-to-end.	1 pt

D — Pipeline CI/CD (5 points)

Critère	Détail d'évaluation	Points
D1 — Workflow 1 présent	Fichier YAML du Workflow 1 présent dans .github/workflows/ (ou équivalent), syntaxe valide.	0.5 pt
D2 — Déclencheur merge main	Workflow 1 déclenché automatiquement sur merge vers main (et non sur push direct). Démontré en live.	1 pt
D3 — Déploiement automatique	Workflow 1 : SSH vers EC2, git pull, docker build, docker restart, health check — tout automatique et fonctionnel.	1 pt
D4 — Workflow 2 présent	Fichier YAML du Workflow 2 présent, déclencheur défini (cron, manual, ou .dvc), syntaxe valide.	0.5 pt
D5 — Retraining automatique	Workflow 2 : dvc repro lancé, nouveau modèle enregistré dans MLflow, API redémarrée — tout automatique.	1 pt
D6 — Secrets CI/CD	Tous les credentials dans GitHub Secrets (ou équivalent). Aucun secret visible dans les fichiers de workflow ni dans le code.	1 pt

E — Structure Git & Collaboration (3 points)

Critère	Détail d'évaluation	Points
E1 — Branches obligatoires	Les trois branches existent : main, staging, dev-MATRICULE pour chaque membre. Historique de merges cohérent.	1 pt
E2 — Commits individuels	Chaque membre a au moins 3 commits significatifs sur sa branche dev-MATRICULE. Messages de commit descriptifs.	1 pt
E3 — .gitignore et secrets	params.yaml et fichiers de secrets exclus du repository via .gitignore. Aucun credential dans l'historique git.	0.5 pt
E4 — README du projet	README.md présent, décrivant le projet, la structure, les instructions de déploiement, et les membres du groupe.	0.5 pt

RÉCAPITULATIF DE LA NOTATION

Section	Intitulé	Points
A	Infrastructure MLflow & AWS	3
B	Pipeline DVC	4

C	API et Interface Utilisateur	4
D	Pipeline CI/CD	5
E	Structure Git & Collaboration	3
	TOTAL PROJET (note de groupe)	**19**
	Assiduité (individuel)	1
	TOTAL FINAL	**20**

4. Pénalités

Infraction	Pénalité sur la note groupe	Plafond
Secret (clé AWS, SSH, mot de passe) trouvé dans le code ou l'historique git	**-1 pts**	cumulable
Utilisation du dataset diabetes.csv des labs	**-3 pts**	non cumulable
Membre sans aucun commit sur sa branche dev-MATRICULE	**-2 pt / membre**	max -3 pts
Branche obligatoire manquante (main, staging ou dev-MATRICULE)	**-0.5 pt / branche**	max -2 pts
API non fonctionnelle lors de la démonstration	**Section C = 0**	—
Workflow CI/CD non démontré en live lors de la soutenance	**Section D = 0**	—
Plagiat entre groupes (ex : meme projet présenté par deux groupes) ou fraude avérée	**Note finale = 0**	définitif

5. Modalités de Rendu et Soutenance

5.1. Ce qui doit être soumis

1. L'URL du repository GitHub public (ou avec accès donné à l'encadrant).
2. L'adresse IP publique de l'EC2 avec les ports MLflow (5000) et API (8000) accessibles.
3. Un fichier README.md à la racine du repository contenant : le sujet du projet, les membres et leurs matricules, les instructions de déploiement, les URLs des services.

5.2. Déroulement de la Soutenance

La soutenance se déroule en deux parties :

Partie 1 — Démonstration live (obligatoire, ~10 minutes)

4. Montrer le MLflow UI avec les expériences et le Model Registry.
5. Montrer l'interface utilisateur de l'application et faire une prédiction en direct.
6. Faire un merge sur main en live et montrer le Workflow CI/CD 1 s'exécuter automatiquement dans l'onglet Actions de GitHub.

7. Déclencher le Workflow 2 (retraining) et montrer le nouveau modèle apparaître dans le MLflow Registry.

Partie 2 — Questions techniques (~5 minutes)

Chaque membre peut être interrogé individuellement sur n'importe quelle partie du projet. L'incapacité à expliquer une section peut entraîner un ajustement de la note individuelle.

> ⚠️ **Condition sine qua non**
>
> Si le Workflow CI/CD ne peut pas être démontré en live lors de la soutenance, la section D est notée 0, quels que soient les fichiers présents dans le repository.
>
> Assurez-vous que l'instance EC2 et le conteneur Docker sont actifs le jour de la soutenance.

5.3. Conseil d'organisation pour les groupes

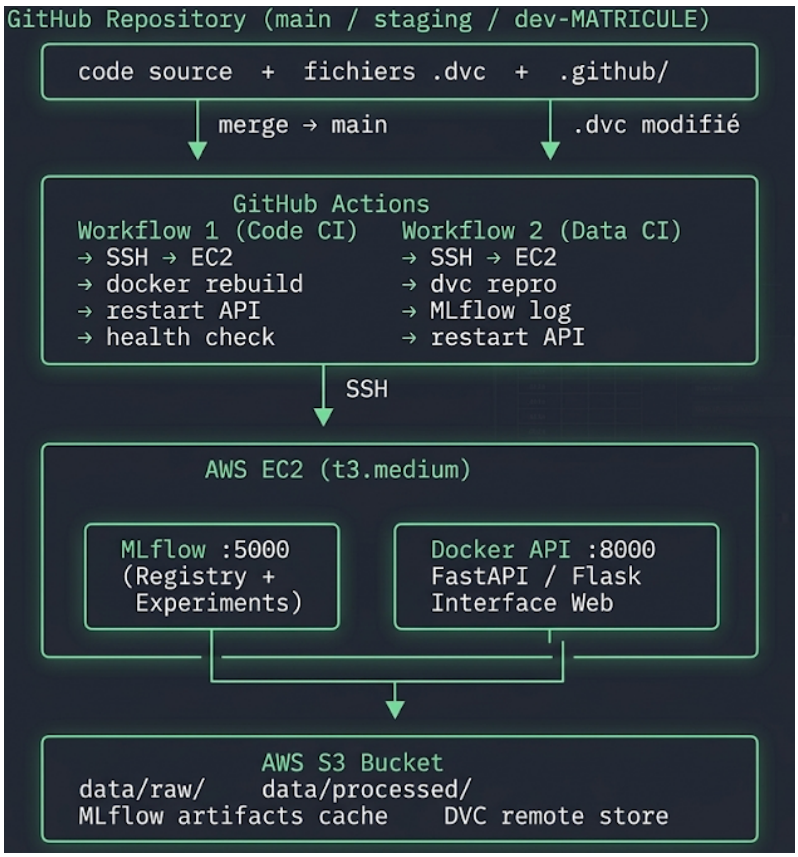
Voici une répartition suggérée des responsabilités (non imposée) :

Profil	Tâches suggérées
ML Engineer	Pipeline DVC (preprocess, train, evaluate), MLflow logging, choix du modèle.
Backend Dev	API FastAPI/Flask, Dockerfile, endpoint /predict, chargement du modèle MLflow.
DevOps / Cloud	EC2, S3, Security Groups, GitHub Secrets, Workflows CI/CD YAML.
Frontend Dev	Interface utilisateur, formulaire de saisie, affichage des prédictions, README.

> 💡 **Rappel**
>
> Cette répartition est indicative — chaque membre doit comprendre l'ensemble du projet pour la partie questions.
>
> Les commits git doivent refléter une contribution réelle de chaque membre, pas uniquement du travail sur une seule partie.

6. Architecture de Référence

L'architecture cible que votre projet doit atteindre est la suivante. Chaque composant correspond à un livrable évalué.


GitHub Repository (main / staging / dev-MATRICULE)
code source + fichiers .dvc + .github/
merge → main
.dvc modifié
GitHub Actions
Workflow 1 (Code CI)
→ SSH → EC2
→ docker rebuild
→ restart API
→ health check
Workflow 2 (Data CI)
→ SSH → EC2
→ dvc repro
→ MLflow log
→ restart API
SSH
AWS EC2 (t3.medium)
MLflow :5000
(Registry +
Experiments)
Docker API :8000
FastAPI / Flask
Interface Web
AWS S3 Bucket
data/raw/
data/processed/
MLflow artifacts cache
DVC remote store

7. Checklist de Vérification Avant Soutenance

Utilisez cette checklist pour vous assurer que rien n'est oublié avant le jour de la soutenance. Chaque item correspond à un critère de la grille de notation.

Infrastructure

- [ ] Instance EC2 démarrée et accessible (ping ou SSH).
- [ ] Serveur MLflow accessible sur http://EC2_IP:5000.
- [ ] Artefacts visibles dans le bucket S3 (dans la console AWS).
- [ ] Au moins un modèle visible dans le MLflow Model Registry.
- [ ] API Docker accessible sur http://EC2_IP:8000.

Pipeline DVC

- [ ] `dvc repro` s'exécute sans erreur depuis zéro.
- [ ] Chaque stage est visible dans dvc.yaml avec ses dépendances et outputs.
- [ ] `dvc push` / `dvc pull` fonctionnels avec le remote S3.
- [ ] Métriques loggées dans MLflow après dvc repro.

API et Interface

- [ ] `curl POST /predict` retourne une prédiction correcte.
- [ ] `curl GET /` retourne HTTP 200.
- [ ] Interface web accessible depuis un navigateur et fonctionnelle.
- [ ] L'API charge bien le 'latest' modèle (changer le modèle dans MLflow et redémarrer l'API pour vérifier).

CI/CD

- [ ] Workflow 1 visible dans GitHub Actions avec historique de runs.
- [ ] Faire un merge test sur main → vérifier que le Workflow 1 se déclenche.
- [ ] Workflow 2 visible et déclenchable (manuel ou automatique).
- [ ] Tous les secrets dans GitHub Secrets — aucun dans le code.

Git et Collaboration

- [ ] Branches main, staging et dev-MATRICULE pour chaque membre présentes.
- [ ] Historique git : chaque membre a des commits sur sa branche.
- `params.yaml` dans le `.gitignore` et absent du repository distant.
- [ ] README.md complet avec membres, sujet, URLs des services.

Bon courage à tous les groupes !

Ce projet est l'occasion de montrer que vous maîtrisez l'intégralité du cycle MLOps, de la donnée brute au modèle en production, en passant par l'automatisation complète.

N'hésitez pas à poser vos questions avant la soutenance — les questions de fond sont les bienvenues, les questions de grille ne le sont pas.

La communication entre membres du groupe et l'organisation du travail font partie de l'évaluation implicite.